

# Week 1 — Production Python

Classes, dataclasses, type hints, Pydantic, pytest, and packages.

The skills that turn a notebook into a working system.

## Why This Week Comes First

- The course builds a cloud AI pipeline over 11 weeks.
- Week 4 turns the model into a reusable component.
- Weeks 9 to 11 connect it to a cloud pipeline.
- All of that needs importable, tested, validated code.

## From Notebook to System

- A notebook confirms code worked once, on one machine.
- Production code is imported, tested, and re-run.
- This week teaches the five tools that make that possible.

## Classes: Bundle Data with Behavior

- A dictionary holds data, but the logic lives elsewhere.
- Nothing connects the two, and typos do not raise errors.
- A class combines data (attributes) with behavior (methods).

# Anatomy of a Class

```
class Thermometer:  
    def __init__(self, location):  
        self.location = location  
        self.readings = []  
  
    def add(self, reading):  
        self.readings.append(reading)
```

## `__init__`, `self`, and `__repr__`

- `__init__` runs when you create the object.
- `self` is the object the method is working on.
- `__repr__` controls how the object prints.
- Without `__repr__` you see only a memory address.

## Configure Once, Apply Many Times

- Some classes hold a configuration and apply it repeatedly.
- Set the thresholds once, then check many records.
- This is the shape of the Week 4 model component.

## Type Hints: Say What You Mean

```
def to_readings(response: WeatherResponse) -> list[HourlyReading]:  
    ...
```

- The signature now states what goes in and comes out.
- Use `list[float]` for containers and `float | None` for optional.



## Type Hints Are Not Enforced

- Python ignores hints at runtime; it does not check them.
- A wrong type fails later, with a confusing message.
- Hints help your editor, type checkers, and readers.

## Dataclasses: Remove the Boilerplate

```
from dataclasses import dataclass

@dataclass
class HourlyReading:
    timestamp: str
    temperature_c: float
    precipitation_mm: float
```

- `@dataclass` writes `__init__`, `__repr__`, and `__eq__`.

## Two Dataclass Rules to Remember

- Mutable defaults need `field(default_factory=list)` .
- A plain `= []` default is shared across all instances.
- `frozen=True` makes an instance read-only and hashable.

## Docstrings: Meaning and Units

- Type hints say what a value is.
- Docstrings say what it means and why it matters.
- Record units, edge cases, and non-obvious choices.

## Pydantic: Validate at the Boundary

- Inside your program, you control the data.
- Outside data (APIs, files, forms) is untrusted.
- The boundary is where outside data becomes inside data.
- Validate there, once, then trust it downstream.

## A Pydantic Model

```
from pydantic import BaseModel, Field

class Reading(BaseModel):
    station_id: str = Field(min_length=3)
    temperature_c: float = Field(ge=-90, le=60)
    humidity: float = Field(ge=0, le=100)
```

- Types are enforced when the object is built.

## What Pydantic Gives You

- Reports every error at once, with the exact location.
- Coerces `"21.5"` to a float; rejects `"very humid"`.
- `Field` adds range and length constraints.
- `model_validator` adds rules that span two fields.

## Dataclass or Pydantic Model?

- Pydantic model: outside data that must be checked.
- Dataclass: inside data your own code produced.
- Validate once at the boundary, then use light objects.



## pytest: Write the Check Down

```
def test_temp_range():  
    day = HourlyReading("2026-04-08T00:00", 20.1, 0.0)  
    assert day.temperature_c == 20.1
```

- A test is a function named `test_*` using plain `assert` .

## pytest Tools You Will Use

- `pytest.approx` for float comparisons; never use `==`.
- `pytest.raises(Error, match=...)` to test failure paths.
- `@pytest.mark.parametrize` for many cases in one test.
- Fixtures for setup shared across tests.

## A Test That Cannot Fail Is Useless

- It creates false confidence while checking nothing.
- After writing a test, break the code on purpose.
- Confirm the test fails, then fix the code.

# Modules and Packages

- A module is one `.py` file.
- A package is a directory of modules with `__init__.py`.
- Split code by cohesion: things that change together.
- `__init__.py` defines the public entry point.

## Scripts vs. Libraries

- A library defines things and does nothing on import.
- A script performs work when you run it.
- Guard the work with `if __name__ == "__main__":`.
- Put the work in `main()` so it stays importable.

## The `conftest.py` Detail

- Tests import the package like any other caller.
- A bare `pytest` searches from the test directory.
- An empty `conftest.py` at the root fixes the import path.
- Without it: `ModuleNotFoundError: No module named 'weatherkit'`.

## Where This Leads: Week 4

- This week builds the `weatherkit` package.
- Week 4 builds a `WeatherClassifier` the same way.
- A loaded model in `__init__`, a `predict()` method, tests.
- The package layout you learn now is the layout you reuse.